

PORTADA

Técnicas de Programación

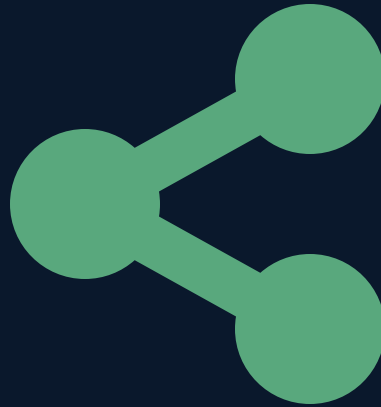
Unidad 2 — Paradigma Objetual: Interfaces y Polimorfismo (Parte 1)

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Un ave y un avión no comparten un ancestro común.
Pero ambos pueden prometer lo mismo: "yo sé volar".**



TRANSICIÓN

Cuando ni siquiera hay una superclase en común

La clase abstracta que viste la sesión pasada resuelve bien el caso de `Figura`, `Circulo` y `Cuadrado`, porque las tres comparten una misma familia. Pero `Ave` y `Avion` no tienen nada más en común — no tendría sentido forzarlos a heredar de una misma superclase solo para compartir `volar()`. Java resuelve esto con un mecanismo distinto: la **interfaz**.

CONCEPTO

Interfaz: un contrato, no una familia

Una **interfaz** declara **qué** métodos debe tener una clase, sin decir **cómo** los implementa — es un contrato puro. A diferencia de la herencia (que modela "es un/a" dentro de una misma familia de clases), implementar una interfaz solo promete: "**esta clase sabe hacer esto**", sin importar su origen.

```
public interface Volador {  
    void volar();  
}
```

CONCEPTO

Sintaxis: interface e implements

Una clase se conecta a una interfaz con `implements` (no `extends` , que es exclusivo de herencia de clases) y está obligada a implementar **todos** los métodos que la interfaz declara.

```
public class Ave implements Volador {
    @Override
    public void volar() {
        System.out.println("El ave bate sus alas para volar.");
    }
}

public class Avion implements Volador {
    @Override
    public void volar() {
        System.out.println("El avión usa sus motores para volar.");
    }
}
```

CONTRA EJEMPLO

Lo que una interfaz no puede tener

A diferencia de una clase (abstracta o no), una interfaz **no puede declarar atributos de instancia** — solo constantes, que son `public static final` de forma implícita, aunque no se escriban esas palabras.

```
public interface Volador {  
    int ALTURA_MAXIMA_METROS = 12000; // implícitamente public static final  
  
    // private int altitudActual; // Error: una interfaz no guarda estado propio  
    void volar();  
}
```

CONCEPTO

Una clase, varias interfaces

Java permite heredar de **una sola** superclase (`extends`), pero implementar **tantas interfaces como necesite** (`implements` separadas por comas). Es una de las razones por las que las interfaces son tan usadas: dan flexibilidad que la herencia simple no permite.

```
public interface Nadador {
    void nadar();
}

public class Pato extends Ave implements Nadador {
    @Override
    public void nadar() {
        System.out.println("El pato nada en la superficie.");
    }
    // volar() ya lo hereda de Ave, que implementa Volador
}
```

TENSIÓN CONCEPTUAL

¿Clase abstracta o interfaz?

No son intercambiables — cada una responde a una pregunta distinta de diseño.

	Clase abstracta	Interfaz
Relación que modela	"Es un/a" (misma familia)	"Sabe hacer esto" (capacidad, sin importar la familia)
Puede tener estado (atributos de instancia)	Sí	No
Puede tener métodos con implementación	Sí	No (en este curso — hay una excepción avanzada con <code>default</code> que no cubrimos aquí)
Cuántas puede usar una misma clase	Una sola superclase	Varias interfaces a la vez



CONCEPTO

Polimorfismo: una palabra, muchas formas

Polimorfismo (del griego "muchas formas") es la capacidad de tratar objetos de tipos distintos **de manera uniforme**, a través de un tipo común (una superclase o una interfaz), dejando que cada objeto responda **a su propia manera** cuando se le pide lo mismo.

Ya viste su base en la sesión de herencia: cuando `Profesor` sobrescribía `saludar()` con `@Override`, eso era polimorfismo en miniatura. Hoy lo llevamos a su forma completa, combinado con interfaces.

CONCEPTO

Upcasting: una referencia de tipo más general

Una variable declarada del tipo de una interfaz (o de una superclase) puede referenciar un objeto de **cualquier clase que la implemente**. A esto se le llama *upcasting*, y Java lo hace automáticamente.

```
Volador v1 = new Ave();      // una referencia Volador apunta a un objeto Ave
Volador v2 = new Avion();    // la misma referencia, apuntando a un Avion

v1.volar(); // "El ave bate sus alas para volar."
v2.volar(); // "El avión usa sus motores para volar."
```

El método que realmente se ejecuta depende del objeto real, no del tipo de la variable — esto se decide en tiempo de ejecución, igual que con `@Override`.

CONCEPTO

Polimorfismo en acción: una colección mixta

La utilidad real del polimorfismo aparece al recorrer una colección de objetos de tipos distintos **a través de un mismo tipo de referencia**, sin necesitar un `if` por cada tipo concreto.

```
Volador[] voladores = { new Ave(), new Avion() };

for (Volador v : voladores) {
    v.volar(); // cada uno ejecuta SU propia versión de volar()
}
// "El ave bate sus alas para volar."
// "El avión usa sus motores para volar."
```



CONCEPTO

Downcasting: volver al tipo específico

Si necesitas usar un método que **no** está en la interfaz (por ejemplo, algo exclusivo de `Avion`), debes convertir la referencia explícitamente de vuelta a su tipo concreto — esto se llama ***downcasting***, y a diferencia del upcasting, **no es automático**.

```
Volador v = new Avion();  
Avion a = (Avion) v;      // downcasting explícito  
a.aterrizarEnPista();     // método propio de Avion, no de Volador
```

Si el objeto real no fuera del tipo al que intentas convertirlo, Java lanza un error en tiempo de ejecución (`ClassCastException`) — por eso el downcasting se usa con cuidado, y casi siempre después de verificar el tipo.

INTERACCIÓN

Diseñen el contrato compartido

En parejas: definan una interfaz `Pagable` con el método `calcularMonto()` , y dos clases sin relación de herencia entre sí que la implementen (ej. `Factura` y `Nomina`). Luego describan cómo recorrerían un arreglo mixto de ambas usando polimorfismo.

6 minutos · estén listos para explicar por qué usaron interfaz y no herencia

SÍNTESIS

Lo que hay que llevarse de hoy

1. Una interfaz es un contrato puro: declara qué métodos debe tener una clase, sin estado propio ni implementación.
2. Una clase puede implementar varias interfaces a la vez, aunque solo pueda extender una única superclase.
3. El polimorfismo permite tratar objetos de tipos distintos de forma uniforme a través de un tipo común, y cada uno responde a su propia manera en tiempo de ejecución.
4. El upcasting es automático; el downcasting es explícito y puede fallar si el objeto no es del tipo esperado.



CIERRE

Antes de la próxima sesión

- Implementar en el IDE la interfaz `Volador` con `Ave` y `Avion`, y recorrer un arreglo mixto comprobando que cada objeto ejecuta su propia versión de `volar()`.
- Traer dudas sobre cuándo usarían interfaz vs. clase abstracta en un caso propio — se retoma con más ejemplos en la próxima sesión.

La próxima sesión es la Parte 2 de este tema: más ejemplos de polimorfismo aplicado, antes de pasar a entrada y salida de información.

